

网络流

基础知识简介



问题模型

- ◆ s地往t地运输原料，已知s、t分别是有向图G中的两点，每条有向边都有一定的流量 C_{ij} 。 C_{ij} 表示从i点到j点运输不能超过的运输量。图G可以称作一个网络，而s能够运输到t的最大运输量则是该网络的最大流
- ◆ 这里s和t分别称为源点和汇点

可行流

- ◆ 在网络的一个可行流中，即任意一个实际运输情况中，实际运输量必须满足一些关系：
- ◆ 1—实际运输量不能是负的
- ◆ 2—每条弧的实际运输量不能大于该弧的容量
- ◆ 3—除了起点 V_s 和重点 V_t 外，对其他定点来说指向 V_i 的弧上的运输量之和，应该等于所有从 V_i 出发的弧上的运输量之和

可改进路P

- ◆ 若给定一个可行流 $F=(F_{ij})$,我们把网络中 $F_{ij}=C_{ij}$ 的弧称作饱和弧, $F_{ij}<C_{ij}$ 的弧称作非饱和弧, $F_{ij}=0$ 的弧称作零流弧, $F_{ij}>0$ 的弧称作非零流弧
- ◆ 若P是网络中联结源点s和汇点t的的一条路(不用管边的有向性), 我们定义路的方向是从 V_s 到 V_t , 则路上的弧被分为两类——一类与路的方向一致, 称为前向弧; 另一类和路的方向相反, 称为后向弧

可改进路P

- ◆ 如果对于P上的每一条前向弧都是非饱和弧，每一条后向弧都是非零流弧，则称P为一条可改进路。因为可以通过修正P上各弧的流量来使得总流量变得更大。修正的方法是：
- ◆ 1—不属于P的弧一概不变
- ◆ 2—对于P上的所有前向弧加上 a ，后向弧减去 a 。这里 a 是一个可改进量。

求网络的最大流

- ◆ a 既要尽量大，又要保证变化后 $0 \leq F_{ij} \leq C_{ij}$ 。
因此 $a = \min(\min(C_{\text{前向弧}ij} - F_{\text{前向弧}ij}), \min(F_{\text{后向弧}ij}))$ 。
- ◆ 如果不存在 V_s 到 V_t 的可改进路，则该网络已经形成了最大流。
- ◆ 那么求最大流的算法可以大致表示为：不断寻找可改进路 P ，并修改 P ，直到找不到 P 为止。这和用匈牙利算法求二分图最大匹配的思路是一致的。
- ◆ 下面介绍一种用标号法来寻找可改进路的算法。

求最大流的标号法

- ◆ 每个顶点的标号分为两个部分：记录是否标号的部分和记录是否检查的部分。
- ◆ 标号法流程：
 - ◆ 1—先标记源点s为已标号未检查
 - ◆ 2—找出一个已标号未检查的点i
 - ◆ 3—寻找所有未标号的和i相连的点j，弧 F_{ij} 必须满足可改进路中对弧的要求。记j为已标号未检查。
 - ◆ 4—记i为已标号已检查，并重复2—4的过程，直到汇点t已标号。



求最大流的标号法

- ◆ 由此我们得到了一条有s到t的可改进路。
- ◆ 先修改可改进路，然后重复上述标号过程，直到找不出可改进路为止。
- ◆ 最后我们便得到了整个网络的最大流。
- ◆ 至于可改进路的记录和改进量 a 的计算等，可以在标号的过程中同时完成，这里可以参考程序。

最大流参考程序

- ◆ Function ford : boolean;
- ◆ var i,j,a : integer;
- ◆ begin
- ◆ ford := false;
- ◆ fillchar(p,sizeof(p),0);
- ◆ p[s].pre := s;
- ◆ repeat
- ◆ for i := 1 to n do if (p[i].pre<>0)and(p[i].check=0) then break;
- ◆ if Not((p[i].pre<>0)and(p[i].check=0)) then exit;
- ◆ for j := 1 to n do if (p[j].pre=0)and(c[i,j]<>0)or(c[j,i]<>0) then
- ◆

最大流参考程序

- ◆ begin
- ◆ if ($f[i,j] < c[i,j]$) then $p[j].pre := i$;
- ◆ if ($f[j,i] > 0$) then $p[j].pre := -i$;
- ◆ end;
- ◆ $p[i].check := 1$;
- ◆ until $p[t].pre \neq 0$;
- ◆ $i := t$; $a := \maxint$;
- ◆ repeat
- ◆ $j := i$; $i := \text{abs}(p[j].pre)$;
- ◆ if $p[j].pre < 0$ then if ($f[j,i] < a$) then $a := f[j,i]$;
- ◆ if $p[j].pre > 0$ then if ($c[i,j] - f[i,j] < a$) then $a := c[i,j] - f[i,j]$;
- ◆ until $i = s$;
- ◆

最大流参考程序

- ◆ $i := t;$
- ◆ repeat
- ◆ $j := i; i := \text{abs}(p[j].\text{pre});$
- ◆ if $p[j].\text{pre} < 0$ then $f[j,i] := f[j,i] - a;$
- ◆ if $p[j].\text{pre} > 0$ then $f[i,j] := f[i,j] + a;$
- ◆ until $i = s;$
- ◆ $\text{ford} := \text{true};$
- ◆ end;
- ◆ 主程序中调用 `while ford do;` 即可求得该网络的最大流。



容量有上下界的网络

- ◆ 对于弧上的流量我们做以下限制： $B(e) \leq F(e) \leq C(e)$ ，在这种情况下，网络中不一定有可行流存在。例如：
- ◆ $s \rightarrow a \rightarrow t$ 其中 $s \rightarrow a$ 的流量限制为 $[0, 1]$ ， $a \rightarrow t$ 的流向限制为 $[2, 3]$ ，这样的网络不存在可行流
- ◆ 那么如何判断一个容量有上下界的网络是否存在可行流呢？很容易想到一种思路——将容量有上下界的网络 N 转换称为一个只有容量上限的，即下限为 0 网络 N' ，然后求 N' 的最大流，根据 N' 的最大流情况可以判断 N' 是否有可行流



求容量有上下界网络的可行流

- ◆ 具体的转换方法如下：
- ◆ 1—新增两个顶点 s' 和 t' ，分别称为附加源点和附加汇点
- ◆ 2—对于原网络 N 的每个顶点 U ，加一条新弧 $e=Ut'$ ，这条弧的容量为所有以 U 为尾的弧的容量的下限之和
- ◆ 3—对于原网络 N 的每个顶点 U ，加一条新弧 $e=Us'$ ，这条弧的容量为所有以 U 为头的弧的容量的下限之和



求容量有上下界网络的可行流

- ◆ 4—原网络 N 的每条弧仍然保留，弧的容量修正为 $C(e)-B(e)$
- ◆ 5—再添两条新弧 $e=st$ ， $e'=ts$ 。 e' 和 e 的容量都为无穷大。
- ◆ 转换好网络之后，求出转换后网络 N' 的最大流，若结果能使流出 s' 的一切弧皆满载则原网络 N 存在可行流，否则不存在可行流。



求容量有上下界网络的最大流和最小流

- ◆ 如果可行流存在，那么在原网络 N 中令 $F(e)=F'(e)+B(e)$ ，由此得到了一个原网络 N 的可行流的具体情况。再用标号法求得该网络的最大流即可。
- ◆ 这里每一条可改进路中，可改进量 a 的取值是： $a=\min(\min(C_{\text{前向弧}ij}-F_{\text{前向弧}ij}), \min(F_{\text{后向弧}ij}-F_{\text{后向弧}ij}))$
- ◆ 求最小流的方法和最大流的方法基本一致，只是在已经求得一个可行流后在对 N 求最大流时，将源点和汇点反一下，这样就变成了一个最小流的算法。



最小费用最大流

- ◆ 在实际生活中，涉及“流”的问题时，人们参考的还不只是流量，还有“费用”的因素。我们研究的便是再保证流量最大的情况下，怎样使得总费用最小。
- ◆ 也就是说，在图 G 中的每一条弧，除了有流量上限之外，还有每单位流量的费用。我们需要求的是最小费用最大流。

最小费用最大流

- ◆ 寻找最大流的方法是从某个可行流出发，找到关于这个可行流的一条可改进路 P 沿着 P 调整流量 F ，对新的可行流试图再寻找它的可改进路，如此反复得到最大流。现在要寻求最小费用的最大流，我们发现，在改进一条可改进路的时候，对于总费用的影响是加上了 P 上所有前向弧的单位费用和与所有后项户单位费用差的差乘以改进量 a 这样一个只。我们称这个值为 P 的改进费用。

最小费用最大流

- ◆ 求最小费用最大流的算法轮廓则是：每次找出一条改进费用最小的可改进路 P ，然后延 P 修改流量，对新的可行流试图再次寻找费用最小的可改进路，如此重复得到最小费用最大流。
- ◆ 这个算法的关键在于如何求得改进费用最小的可改进路。我们建立一个新的赋权图 W ，定义 W 中的每一条弧的权 W_{ij} 和 W_{ji} 为一
- ◆ If $F_{ij} < C_{ij}$ then $W_{ij} := B_{ij}$ else $W_{ij} := \text{无穷大}$;
- ◆ If $F_{ij} > 0$ then $W_{ji} := -B_{ij}$ else $W_{ji} := \text{无穷大}$



最小费用最大流

- ◆ 要在原图中寻找费用最小的可改进路，相当于在有向图 W 中寻找由源点 s 到汇点 t 的最短路。
- ◆ 这里计算 W 中的最短路是否能用dijkstra？简单地说，是在一个有负权边的有向图中求最短路。
- ◆ 这里是不能用dijkstra的，只能用bellman。



最小费用最大流参考程序

- ◆ Function ford : boolean;
- ◆ begin
- ◆ ford := false;
- ◆ for i := 1 to n do d[i] := maxint;
- ◆ fillchar(pre,sizeof(pre),0);
- ◆ pre[s] := s; f[s] := 0;
- ◆ repeat
- ◆ begin
- ◆ find := true;



最小费用最大流参考程序

- ◆ for i := 1 to n do if d[i] <> maxint then
- ◆ for j := 1 to n do if i <> j then if g[i,j].c > g[i,j].f then
- ◆ if d[i] + g[i,j].w < d[j] then
- ◆ begin
- ◆ d[j] := d[i] + g[i,j].w;
- ◆ pre[j] := i;
- ◆ find := false;
- ◆ end;
- ◆ end;



最小费用最大流参考程序

- ◆ until find;
- ◆ if $\text{pre}[t] = 0$ then exit;
- ◆ ford := true;
- ◆ j := t; d := maxint;
- ◆ repeat
- ◆ i := pre[j];
- ◆ if $g[i,j].c - g[i,j].f < d$ then $d := g[i,j].c - g[i,j].f$;
- ◆ j := i;
- ◆ until j = s;



最小费用最大流

- ◆ $j := t;$
- ◆ repeat
- ◆ $i := \text{pre}[j];$
- ◆ $\text{inc}(g[i,j].f, d);$
- ◆ $g[j,i].f := -g[i,j].f;$
- ◆ $j := i;$
- ◆ until $j = s;$
- ◆ 主程序中调用while ford do;就可以了